

Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования

«Российский государственный университет имени А. Н. Косыгина
(Технологии. Дизайн. Искусство)»

Институт информационных технологий и цифровой трансформации

наименование института

Искусственного интеллекта, прикладной математики и программирования

наименование кафедры

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА
БАКАЛАВРА

на тему: Сервисная платформа хранения данных о медицинских препаратах

наименование темы выпускной квалификационной работы

Направление подготовки: 01.03.02 Прикладная математика и информатика

код, наименование направления подготовки

Профиль: Системное программирование и компьютерные технологии

Выполнил

студент группы МПМ-122 4 курса очной формы обучения

очной, очно-заочной, заочной

Белявцев Владислав Сергеевич

подпись

фамилия, имя, отчество

Руководитель:

старший преподаватель,
Костоев А. Т.

подпись

ученая степень, ученое звание, фамилия, инициалы

«Допущена к защите»

Заведующий кафедрой
искусственного интеллекта,
прикладной математики и
программирования

к.ф-м.н., Мокряков А. В.

подпись

Москва 2026

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	5
1. АНАЛИЗ И ОБОСНОВАНИЕ ПОДХОДА.....	7
1.1. Предметная область и требования.....	7
1.2. Обзор архитектурных стилей.....	9
1.3. Обзор способов клиент-серверного взаимодействия	11
1.4. Обзор подходов к хранению, кэшированию и обработке данных.....	13
1.5. Обзор подходов к обеспечению безопасности	15
1.6. Обзор средств контейнеризации и оркестрации	18
1.7. Обзор аналогов	19
2. ПОДХОДЫ И МОДЕЛЬ СИСТЕМЫ.....	22
2.1. Общая архитектура	22
2.2. Модель предметной области	23
2.3. Проектирование программного интерфейса	27
2.4. Проектирование безопасности.....	29
2.5. Проектирование обработки данных	31
2.6. Проектирование кэширования и ограничения нагрузки	33
2.7. Проектирование хранения данных	35
2.8. Проектирование развертывания и масштабирования.....	37
3. ПРАКТИЧЕСКАЯ РЕАЛИЗАЦИЯ	40
3.1. Организация программного решения.....	40
3.2. Реализация слоя данных	40
3.3. Реализация прикладной логики	42
3.4. Реализация API и перехватчиков	43

3.5. Реализация безопасности	44
3.6. Реализация доставки и фоновых задач	45
3.7. Реализация кэширования и ограничения нагрузки	47
3.8. Контейнеризация и оркестрация.....	48
3.9. Тестирование	49
ЗАКЛЮЧЕНИЕ.....	51
СПИСОК ИСТОЧНИКОВ	54
ПРИЛОЖЕНИЕ А.....	56

ВВЕДЕНИЕ

Развитие цифровой медицины и фармацевтики сопровождается стремительным ростом объема и сложности данных, описывающих лекарственные препараты, биологически активные добавки, диагнозы, побочные эффекты и индивидуальные особенности приема у конкретного пациента. Сведения о препаратах перестают существовать изолированно: они образуют связанные структуры данных, в которых диагноз сопоставляется с возможными схемами лечения, препарат – с сопутствующими добавками и потенциальными нежелательными реакциями, а индивидуальная история приема – с журналами самонаблюдения пользователя. Накопление, структурирование и безопасная выдача таких данных становятся самостоятельной задачей, выходящей за рамки простого мобильного приложения-напоминания и требующей построения полноценной серверной платформы хранения.

К платформе хранения медицинских данных предъявляются жесткие нефункциональные требования. От нее ожидается надежность (данные не должны теряться, а связанные операции – нарушаться при частичных сбоях инфраструктуры), масштабируемость (способность обслуживать растущее число пользователей без замедлений отклика) и предсказуемость поведения под нагрузкой.

Существующие медицинские и фармацевтические справочники, а также трекеры приема препаратов, как правило, ориентированы на конечного потребителя и закрывают лишь часть перечисленного спектра задач, недостаточно полно решая вопросы строгой типизации контрактов, надежной доставки служебных сообщений и эксплуатационной устойчивости при отказах отдельных компонентов. Таким образом, существует потребность в проектировании и реализации серверной платформы, которая объединяла бы структурированное хранение справочных и персональных медицинских данных с современными инженерными решениями в области безопасности, надежности и масштабируемости.

Объектом исследования являются серверные платформы хранения медицинских данных, обеспечивающие прием, хранение и предоставление сведений о препаратах и связанной с ними информации о пользователе. Предметом исследования выступают методы и архитектурные подходы к проектированию платформы, включая выбор архитектурного стиля, организацию доступа к данным, обеспечение конфиденциальности персональной медицинской информации и горизонтальное масштабирование.

Цель работы – разработать платформу хранения данных о препаратах, корректно работающую при горизонтальном масштабировании.

Для достижения поставленной цели в работе решаются следующие задачи:

- провести анализ предметной области, сформулировать функциональные и нефункциональные требования к платформе;
- обосновать выбор архитектурного стиля, технологического стека, протокола взаимодействия и модели хранения данных сравнив разные подходы и способы работы;
- спроектировать модель предметной области и программный интерфейс платформы, включая разбиение на сервисы, контракты взаимодействия и модель ошибок;
- реализовать прикладную логику платформы, средства обеспечения безопасности, надежную доставку служебных сообщений и кэширование данных в распределенной среде;
- обеспечить контейнеризацию, оркестрацию и горизонтальное автомасштабирование платформы, а также обновление без простоя;
- выполнить модульное и интеграционное тестирование ключевых компонентов и определить методику проверки масштабируемости платформы под нагрузкой.

1. АНАЛИЗ И ОБОСНОВАНИЕ ПОДХОДА

1.1. Предметная область и требования

Предметная область платформы охватывает структурированное хранение и предоставление двух взаимосвязанных групп данных: справочных сведений о медицинских препаратах и персональных данных пользователя, связанных с приемом этих препаратов. Платформа рассматривается не как клиентское приложение-напоминание, а как серверное хранилище, выдающее структурированные данные через программный интерфейс, что определяет состав сущностей и характер требований.

Справочная часть, или каталог, образует основу предметной области. В нее входят диагнозы, лекарственные препараты, биологически активные добавки и побочные эффекты, а также связи между ними: сопоставление диагноза множеству препаратов, препарата – множеству сопутствующих добавок и препарата – множеству возможных побочных эффектов. Каталог наполняется администратором посредством массового импорта данных из табличных файлов с сохранением истории выполненных импортов.

Персональная часть данных описывает индивидуальную медицинскую картину пользователя. К ней относятся профиль пользователя, назначенные ему диагнозы, принимаемые препараты и добавки. Отдельную группу образуют журналы самонаблюдения: журнал побочных эффектов с градацией интенсивности по уровням от низкой до тяжелой, журнал самостоятельно добавленных пользователем «внешних» препаратов, отсутствующих в централизованном каталоге, и записи менструального цикла с градацией интенсивности.

Пользователями платформы выступают две роли. Пациент работает с собственными персональными данными и журналами, а также обращается к справочнику препаратов. Администратор отвечает за наполнение и сопровождение каталога, в том числе выполняет импорт справочных данных

и административные операции. Разграничение ролей определяет необходимость управления доступом на уровне отдельных операций API.

На основании анализа предметной области сформулированы функциональные требования к платформе. Во-первых, платформа должна хранить и предоставлять каталог препаратов с учетом всех связей между диагнозами, препаратами, добавками и побочными эффектами. Во-вторых, она должна обеспечивать ведение персональных данных пользователя: профиля, назначенных диагнозов, принимаемых препаратов и добавок. В-третьих, требуется ведение журналов самонаблюдения с фиксацией интенсивности и времени событий. В-четвертых, необходима поддержка административного наполнения каталога посредством импорта данных с сохранением истории. В-пятых, платформа должна обеспечивать регистрацию, аутентификацию и управление учетными записями пользователей.

Нефункциональные требования определяются характером обрабатываемых данных и условиями эксплуатации. Ключевым является требование конфиденциальности и безопасности. Федеральный закон № 152-ФЗ определяет персональные данные как «любую информацию, относящуюся к прямо или косвенно определенному или определяемому физическому лицу (субъекту персональных данных)» [20]. Сведения о состоянии здоровья отнесены этим законом к специальной категории персональных данных, обработка которой допускается лишь при соблюдении дополнительных требований [20]. Это обуславливает необходимость надежной аутентификации, разграничения доступа, защиты учетных данных и противодействия типовым угрозам. Требование надежности означает, что данные не должны теряться, а связанные операции должны сохранять целостность при частичных сбоях инфраструктуры, включая временную недоступность вспомогательных служб.

Требование масштабируемости предполагает способность платформы обслуживать растущее число пользователей за счет горизонтального

наращивания вычислительных ресурсов без существенной деградации времени отклика. Также, к платформе предъявляются требования сопровождаемости и наблюдаемости: структурированное логирование и сквозная трассировка запросов необходимы для эксплуатации и диагностики в среде.

1.2. Обзор архитектурных стилей

Выбор архитектурного стиля определяет способ разбиения системы на части, направление зависимостей между ними, сопровождаемость, тестируемость и масштабируемость решения. В современной инженерии программного обеспечения сложилось несколько базовых подходов к организации серверных приложений. Эти подходы можно сравнить, чтобы понять, какой из них лучше всего подходит для создания системы хранения медицинских данных.

Монолитная архитектура предполагает, что вся функциональность системы разворачивается как единое приложение с общим процессом исполнения. Достоинствами такого подхода являются простота разработки и развертывания на начальных этапах, отсутствие издержек на межсервисное взаимодействие и единая транзакционная граница, упрощающая обеспечение целостности данных. Вместе с тем при неконтролируемом росте монолит склонен превращаться в сильно связанную систему, в которой изменения в одной части провоцируют непредвиденные последствия в других, а раздельное масштабирование отдельных функций оказывается невозможным [3].

Микросервисная архитектура, напротив, предполагает разбиение системы на множество небольших самостоятельно разворачиваемых служб, каждая из которых отвечает за ограниченную область ответственности и взаимодействует с прочими по сети [3]. С. Ньюмен определяет работу микросервисов как «Наш микросервис является самостоятельным образованием, которое может быть развернуто в качестве обособленного

сервиса на платформе, предоставляемой в качестве услуги, — Platform as a Service (PAAS), или может быть процессом своей собственной операционной системы.» [3, с. 24]. Такой подход обеспечивает независимое масштабирование и развертывание отдельных служб, технологическую гетерогенность и организационную изоляцию команд. Однако он влечет существенное усложнение эксплуатации: распределенные транзакции, согласованность данных между службами, сетевые отказы и накладные расходы на взаимодействие становятся постоянными источниками сложности [4, 5]. Для рассматриваемой системы, которая собирает и хранит медицинские данные, разделение на отдельные части может усложнить работу и не принести большой пользы.

Рациональное промежуточное положение занимает модульный монолит — единое развертываемое приложение, в котором, в отличие от неструктурированного монолита, сознательно поддерживаются строгие внутренние границы между функциональными модулями, выделенными по областям предметной области: аутентификация и учетные записи, каталог препаратов, персональные назначения пользователя, журналы самонаблюдения, административный импорт. Каждый модуль обладает высокой внутренней связностью и слабой связанностью с прочими, а взаимодействие осуществляется через явно объявленные интерфейсы. Тем самым модульный монолит переносит во внутреннее устройство единого приложения принцип декомпозиции по бизнес-возможностям, свойственный микросервисам, но сохраняет единый процесс исполнения, единую транзакционную границу и отсутствие сетевых издержек [3, 5].

Логическую организацию кода внутри платформы, независимо от выбранной архитектуры, задают принципы чистой архитектуры, сформулированные Р. Мартином [1], и предметно-ориентированное проектирование Э. Эванса [2]. Чистая архитектура предполагает разделение кода на слои с разными уровнями ответственности: предметная область, прикладная логика, доступ к данным, доставка запросов и однонаправленный

поток зависимостей. Центральной ее идеей является принцип инверсии зависимостей: бизнес-правила и сущности предметной области образуют независимое ядро системы, не зависящее ни от деталей хранения данных, ни от средств доставки запросов, ни от внешних библиотек, тогда как все технические детали реализуются во внешних слоях и зависят от ядра, а не наоборот [1]. Само правило зависимостей Р. Мартин формулирует так: «Зависимости в исходном коде должны быть направлены внутрь, в сторону высокоуровневых политик.» [1, с. 204]. Такой направленный внутрь поток зависимостей обеспечивает устойчивость наиболее ценной части системы – бизнес-логики – к изменениям инфраструктуры, упрощает замену технических компонентов и делает ядро доступным для модульного тестирования в изоляции.

Проанализировав рассмотренные подходы, можно сделать вывод. Выбран модульный монолит, так как полная микросервисная декомпозиция избыточна и сопряжена с неоправданной эксплуатационной сложностью, а неструктурированный монолит не удовлетворяет требованиям сопровождаемости и тестируемости. Логическую организацию занимает чистая архитектура с разделением на слои. Модульный монолит, который организован по принципам чистой архитектуры сочетает простоту развертывания и целостность данных с высокой сопровождаемостью и тестируемостью ядра. При этом единое приложение остается пригодным к горизонтальному масштабированию посредством запуска нескольких идентичных реплик за балансировщиком нагрузки.

1.3. Обзор способов клиент-серверного взаимодействия

Поскольку платформа предоставляет структурированные данные клиентам через API, выбор способа клиент-серверного взаимодействия и протокола передачи непосредственно влияет на строгость контрактов, производительность и удобство разработки клиентов. В практике построения серверных приложений сложились два основных подхода – взаимодействие в

стиле REST и взаимодействие на основе gRPC, которые целесообразно сопоставить применительно к задаче работы.

Архитектурный стиль REST основан на представлении предметной области в виде ресурсов, адресуемых единообразными идентификаторами, и использовании методов протокола HTTP для операций над ними. В качестве формата представления данных, как правило, применяется текстовый JSON. Достоинствами REST являются повсеместная распространенность, простота отладки за счет человекочитаемого формата, естественная поддержка кэширования на уровне HTTP и низкий порог входа для клиентов, включая браузерные. Вместе с тем стиль REST в базовом виде не предписывает строгого машиночитаемого контракта: схема данных и состав полей описываются внешними соглашениями, а не являются неотъемлемой частью протокола, что повышает риск рассогласования клиента и сервера. Текстовая сериализация увеличивает объем передаваемых данных и затраты на разбор, а слабая типизация переносит часть ошибок со стадии компиляции на стадию исполнения.

В официальной документации gRPC определяется как «a modern open source high performance Remote Procedure Call (RPC) framework» [10]. Подход gRPC основан на удаленном вызове процедур поверх протокола HTTP/2 с описанием контрактов в Protocol Buffers. Протокол HTTP/2 вводит бинарное кадрирование, уплотнение множества логических потоков в рамках одного соединения и сжатие заголовков, что снижает задержки и накладные расходы по сравнению с HTTP/1.1 [9]. Protocol Buffers задает схему сообщений и сервисов в декларативных файлах описания, на основе которых автоматически генерируется типизированный код клиента и сервера. Бинарная сериализация обеспечивает компактное представление и быстрый разбор данных, а нумерация полей поддерживает обратную и прямую совместимость контрактов при их эволюции [11]. Сам gRPC предоставляет строго типизированные контракты сервисов, потоковую передачу и единообразную

модель статусов и ошибок [10]. Платформа ASP.NET Core располагает встроенными средствами размещения gRPC-сервисов [17].

Сравнение подходов взаимодействия приводит к выбору gRPC. Платформа выдает структурированные, сильно типизированные данные с многочисленными связями, и строгий машиночитаемый контракт, заданный схемой Protocol Buffers, исключает класс ошибок рассогласования и служит единым источником истины для клиента и сервера. Бинарная сериализация повышают эффективность передачи под нагрузкой, что сочетается с требованием масштабируемости, а автоматическая генерация кода снижает трудоемкость разработки и вероятность ошибок. Известное ограничение gRPC – меньшая пригодность для прямого вызова из браузера и меньшая прозрачность для отладки по сравнению с текстовым REST незначительна в данном случае, поскольку платформа представляет собой серверную часть, обслуживающую программных клиентов, а для отладки предусмотрены вспомогательные механизмы.

1.4. Обзор подходов к хранению, кэшированию и обработке данных

Платформа оперирует структурированными данными с большим числом связей между сущностями справочника и персональными данными пользователя, что определяет требования к подсистеме хранения. Для таких данных естественным выбором является реляционная база данных, обеспечивающая декларативное описание связей, контроль ссылочной целостности и транзакционные гарантии атомарности, согласованности, изоляции и долговечности [4]. Для доступа к данным приложение использует средство объектно-реляционного отображения (ORM). Оно автоматически преобразует объекты предметной области в строки таблиц базы данных и обратно, поэтому прикладная логика работает с привычными объектами и не зависит от того, как именно строятся запросы и как устроена схема хранения [16]. Поверх ORM организуется слой репозитория – отдельный набор классов, который собирает в себе все операции чтения и записи данных и

предоставляет прикладной логике простой интерфейс для обращения к хранилищу. Такой способ организации доступа к данным описан М. Фаулером среди типовых шаблонов корпоративных приложений [6].

В распределенной среде, где платформа выполняется несколькими репликами, возникает специфическая проблема управления соединениями с базой данных. Установление соединения с реляционной СУБД является сравнительно дорогостоящей операцией, а число одновременных соединений ограничено. При росте количества реплик суммарное число соединений способно исчерпать ресурсы сервера базы данных [13]. Решением служит пул соединений – промежуточный компонент, переиспользующий ограниченное множество физических соединений для обслуживания множества логических запросов от реплик, что снижает накладные расходы и предотвращает исчерпание соединений.

Для снижения задержек и нагрузки на хранилище при многократном чтении редко изменяющихся данных применяется кэширование. В распределенной среде различают локальный кэш в памяти отдельной реплики, обеспечивающий минимальную задержку доступа, и распределенный кэш, разделяемый всеми репликами и обеспечивающий согласованность кэшированных данных между ними. Сочетание этих уровней образует двухуровневый кэш, в котором первый уровень обслуживает наиболее частые обращения, а второй сглаживает различия между репликами и переживает их перезапуск [14]. Кэширование, однако, неизбежно порождает проблему согласованности: при изменении исходных данных кэшированные копии во всех репликах должны быть признаны недействительными, иначе клиенты будут получать устаревшие сведения [4]. В распределенной среде согласованная инвалидация локальных кэшей множества реплик требует координирующего механизма, поскольку прямое оповещение каждой реплики усложняет систему. Распространенным решением является хранение во внешнем разделяемом хранилище признака актуальности данных, например версии, с которой сверяются реплики.

Отдельную задачу составляет надежная обработка операций, затрагивающих внешние службы. Ряд бизнес-операций платформы должен не только сохранить изменения в базе данных, но и инициировать внешнее действие, например отправку письма с подтверждением адреса электронной почты или сбросом пароля. Выполнение этих двух действий вне единой транзакции порождает так называемую проблему двойной записи: при сбое между фиксацией данных и отправкой письма система переходит в несогласованное состояние – изменения сохранены, но письмо не отправлено, либо письмо отправлено, а фиксация не состоялась [5]. Классическим решением этой проблемы является паттерн транзакционного Outbox. К. Ричардсон формулирует суть приема так: «Он использует таблицу БД в качестве временной очереди сообщений. У сервиса, отправляющего сообщения, есть таблица OUTBOX. В рамках транзакции, которая создает, обновляет и удаляет бизнес-объекты, сервис шлет сообщения, вставляя их в эту таблицу. Поскольку это локальная ACID-транзакция, атомарность гарантируется.» [5, с. 133]. Согласно ему сообщение о необходимости внешнего действия записывается в специальную таблицу-«ящик исходящих» в той же транзакции базы данных, что и основная бизнес-операция, благодаря чему сообщение и изменение данных фиксируются атомарно. Отдельный фоновый обработчик впоследствии извлекает накопленные сообщения и доставляет их по назначению, помечая обработанными [5]. Такой механизм гарантирует доставку «хотя бы один раз» даже при временной недоступности внешней службы. При исполнении обработчика несколькими репликами требуется дополнительный механизм, предотвращающий повторный захват одного и того же сообщения разными репликами, а также политика повторных попыток при неустранимых с первого раза сбоях.

1.5. Обзор подходов к обеспечению безопасности

Поскольку платформа обрабатывает специальную категорию персональных данных, требования к которой установлены Федеральным

законом № 152-ФЗ [20], вопросы безопасности должны решаться на архитектурном уровне. К типовым угрозам веб-приложений относятся: нарушение аутентификации, ненадежное хранение учетных данных и ошибки контроля доступа.

Базовой задачей является аутентификация – подтверждение личности пользователя и последующая авторизация, то есть проверка права на выполнение конкретной операции. В распределенной среде, где запросы обслуживаются взаимозаменяемыми репликами, нежелательно хранить сведения о сеансе на стороне сервера, так как это связало бы пользователя с конкретной репликой. Этой задаче отвечает токенная аутентификация на основе JSON Web Token – самодостаточного подписанного токена, который содержит сведения о пользователе и его роли, а также срок действия, и проверяется любой репликой по подписи без обращения к разделяемому хранилищу сеансов [8]. Свойство самодостаточности JWT хорошо согласуется с требованием взаимозаменяемости реплик. Обратной стороной этого свойства является то, что выпущенный токен невозможно отозвать до истечения срока его действия. Для смягчения этого ограничения применяется схема из двух типов токенов: короткоживущий токен доступа предъявляется при каждом запросе, а долгоживущий токен обновления служит для получения новых токенов доступа без повторного ввода пароля и, в отличие от токена доступа, может быть отозван. Тем самым ограничивается окно, в течение которого скомпрометированный токен доступа остается действительным.

Отдельного внимания требует хранение паролей. Хранение паролей в открытом виде недопустимо, однако и применение быстрых криптографических хеш-функций общего назначения недостаточно, поскольку их высокая скорость облегчает подбор пароля перебором. Для хранения паролей применяются специальные адаптивные алгоритмы хеширования с гаммой, намеренно замедляющие вычисление хеша и тем самым затрудняющие массовый перебор. Одним из таких алгоритмов является BCrypt, предложенный Н. Провосом и Д. Мазьером, ключевой особенностью

которого является возможность увеличивать вычислительную стоимость по мере роста производительности оборудования.

Помимо защиты самих учетных данных, необходимо противодействовать атакам на процедуру входа. Защита от перебора реализуется ограничением числа неудачных попыток входа: после превышения порога учетная запись временно блокируется, что делает массовый подбор пароля практически невыполнимым. Самостоятельную угрозу представляют атаки по времени отклика: если при обращении к несуществующей учетной записи система отвечает заметно быстрее, чем при существующей, разница во времени позволяет злоумышленнику установить факт наличия учетной записи. Для устранения этой утечки при отсутствии пользователя выполняется проверка пароля против заранее заготовленного фиктивного хеша, благодаря чему время ответа не зависит от существования записи. Аналогичные предосторожности применяются к одноразовым токенам подтверждения адреса электронной почты и сброса пароля: такие токены должны обладать высокой энтропией, в базе данных целесообразно хранить не сам токен, а его хеш, а сравнение предъявленного значения с сохраненным выполнять в постоянном времени, чтобы исключить утечку информации по времени сравнения.

Разграничение прав реализуется через модель ролей: обычный пользователь работает только с собственными данными, тогда как административные операции, в том числе наполнение каталога, доступны лишь пользователям с ролью администратора. Совокупность перечисленных мер – токенная аутентификация, надежное хранение паролей, защита от перебора и атак по времени, безопасная обработка одноразовых токенов и ролевое разграничение доступа образует целостный подход к безопасности платформы.

1.6. Обзор средств контейнеризации и оркестрации

Ранее были выдвинуты требования к масштабируемости и обновлению без простоев. Эти требования подразумевают наличие инструментов, которые позволяют надежно разворачивать платформу и управлять ее экземплярами. Базовым из таких средств является контейнеризация. Контейнер упаковывает приложение вместе с его зависимостями в переносимый образ, который исполняется единообразно в любой среде, поддерживающей контейнеризацию, обеспечивая изоляцию процессов при существенно меньших накладных расходах по сравнению с полноценными виртуальными машинами. Это устраняет расхождения между средами разработки и эксплуатации и упрощает воспроизводимость развертывания. Чтобы запустить локально сразу несколько связанных между собой служб – само приложение и вспомогательную инфраструктуру (например, базу данных и кэш), используют инструменты, в которых все нужные контейнеры заранее описаны в одном файле. Это позволяет поднять всю инфраструктуру одной командой.

Управление множеством контейнеров в условиях эксплуатации, особенно при горизонтальном масштабировании, требует более развитых средств – систем оркестрации. Наиболее распространенной из них является Kubernetes, который автоматизирует размещение контейнеров по вычислительным узлам, поддержание заданного числа экземпляров, обнаружение и перезапуск отказавших экземпляров, обновление приложения и маршрутизацию трафика [12]. В Kubernetes конфигурация задается описанием желаемого результата, а не пошаговых действий: администратор просто указывает, каким должно быть итоговое состояние системы, например, сколько экземпляров приложения должно работать и с какими параметрами. Такие описания называются манифестами. Получив их, оркестратор сам приводит реальное состояние системы к описанному и затем поддерживает его, в том числе перезапускает экземпляр, если тот отказал. Поскольку конфигурация различается между средами (например, средой разработки и средой эксплуатации), для

управления вариативностью манифестов применяются средства наложения изменений на общую базовую конфигурацию, что позволяет избежать дублирования описаний. Входящий внешний трафик направляется к приложению через специальный компонент маршрутизации входящих запросов.

Ключевым для настоящей работы механизмом является горизонтальное автомасштабирование. Оркестратор способен автоматически изменять число экземпляров приложения в зависимости от наблюдаемых показателей нагрузки, например загрузки процессора и потребления памяти: при росте нагрузки число экземпляров увеличивается, при снижении – уменьшается [12]. Для исключения перерывов в обслуживании при обновлении применяется стратегия поэтапного обновления, при которой новые экземпляры запускаются и проверяются на готовность прежде, чем выводятся из эксплуатации старые; готовность и работоспособность экземпляров определяются специальными пробами. Таким образом, средства контейнеризации и оркестрации обеспечивают воспроизводимое развертывание, автоматическое масштабирование и обновление платформы без простоя.

1.7. Обзор аналогов

Для определения ниши разрабатываемой платформы выполнен сравнительный анализ существующих решений, оперирующих данными о медицинских препаратах. Рассмотренные решения распределяются по трем категориям, каждая из которых представлена характерными образцами.

Первую категорию образуют мобильные приложения для контроля приема препаратов, типичными представителями которых являются MyTherapy, Medisafe и Round Health. Их основная функция сводится к напоминаниям о приеме, ориентированным на отдельного пользователя. Ограничениями этой категории выступают привязка данных к одному устройству, отсутствие справочного каталога, структурированного под диагноз, и слабые средства экспорта накопленных сведений лечащему врачу.

Модель данных таких приложений подчинена задаче напоминания, а связи «диагноз – препараты – добавки – побочные эффекты» в ней, как правило, не выражены.

Вторую категорию составляют государственные медицинские информационные системы, к которым относятся ЕМИАС и системы ведения электронных медицинских карт. Эти системы многопользовательские и располагают высоким уровнем требований к безопасности, обусловленным регуляторными нормами. Вместе с тем они ориентированы на учет деятельности медицинских учреждений и приемов, а не на личный самоконтроль пациента: их назначение и модель данных подчинены задачам учреждения, а не индивидуальному ведению персональных журналов самонаблюдения.

Третью категорию образуют открытые медицинские платформы, характерным представителем которых является OpenMRS. Такие платформы функционально мощны и располагают развитой моделью данных и средствами интеграции. Однако для задачи личного самоконтроля пациента они оказываются тяжелыми и избыточными: их развертывание, сопровождение и освоение сопряжены со значительной сложностью, не оправданной применительно к рассматриваемой задаче.

Проведенное сопоставление показывает, что ни одна из рассмотренных категорий не закрывает в полной мере совокупность поставленных задач. Мобильные трекеры ориентированы на пациента, но привязаны к устройству, лишены каталога под диагнозы и слабо интегрируются с врачом. Государственные системы многопользовательские и защищенные, но предназначены для учреждений и приемов, а не для личного самоконтроля; открытые платформы мощны, но тяжелы и избыточны для данной задачи. Тем самым выявляется незанятая ниша – многопользовательская серверная платформа, сочетающая каталог, структурированный под диагнозы, ведение персональных данных и журналов самонаблюдения пациента и надежность под нагрузкой. Именно эта совокупность свойств определяет

позиционирование разрабатываемой платформы и отличает ее от рассмотренных аналогов.

2. ПОДХОДЫ И МОДЕЛЬ СИСТЕМЫ

2.1. Общая архитектура

Платформа спроектирована как модульный монолит, организованный по принципам чистой архитектуры. Программное решение разделено на четыре слоя-проекта с однонаправленным потоком зависимостей, направленным внутрь, к ядру предметной области (см. Рис. 2.1).

Внутренний слой – слой предметной области (Domain) – содержит сущности и бизнес-правила и не зависит ни от каких внешних библиотек, средств доступа к данным или транспорта. Над ним располагается слой прикладной логики (Application), в котором определены сервисы прикладного уровня, интерфейсы доступа к данным и внешним службам, объекты передачи данных и правила валидации. Слой инфраструктуры (Infrastructure) реализует объявленные в прикладном слое интерфейсы: доступ к данным средствами объектно-реляционного отображения и репозитория, отправку писем, фоновые задачи, кэширование, ограничение частоты запросов и средства безопасности. Внешний слой – слой доставки (Grpc) – является точкой входа в приложение: он размещает gRPC-сервер и сервисы, а также цепочку перехватчиков, обрабатывающих каждый запрос.

Ключевым принципом, связывающим слои, является инверсия зависимостей [1]: прикладной слой объявляет интерфейсы (например, интерфейсы репозитория и внешних служб), а слой инфраструктуры предоставляет их конкретные реализации, благодаря чему направление зависимости по исходному коду оказывается противоположным направлению вызова во время исполнения. Связывание абстракций с реализациями выполняется механизмом внедрения зависимостей: на этапе запуска приложения в контейнере зависимостей регистрируются соответствия между интерфейсами и их реализациями, а компоненты получают свои зависимости через конструкторы. Такой подход обеспечивает независимость ядра от инфраструктуры, упрощает замену технических компонентов и делает

прикладную логику доступной для модульного тестирования с использованием подставных реализаций интерфейсов.

Отдельным сквозным аспектом архитектуры является валидация входных данных, вынесенная в прикладной слой и реализуемая декларативными правилами на основе библиотеки FluentValidation. Это отделяет проверку корректности данных от бизнес-логики сервисов и от транспортного слоя и обеспечивает единообразную обработку ошибок валидации. Сквозными также являются структурированное логирование и трассировка запросов по сквозному идентификатору корреляции, пронизывающие все слои.

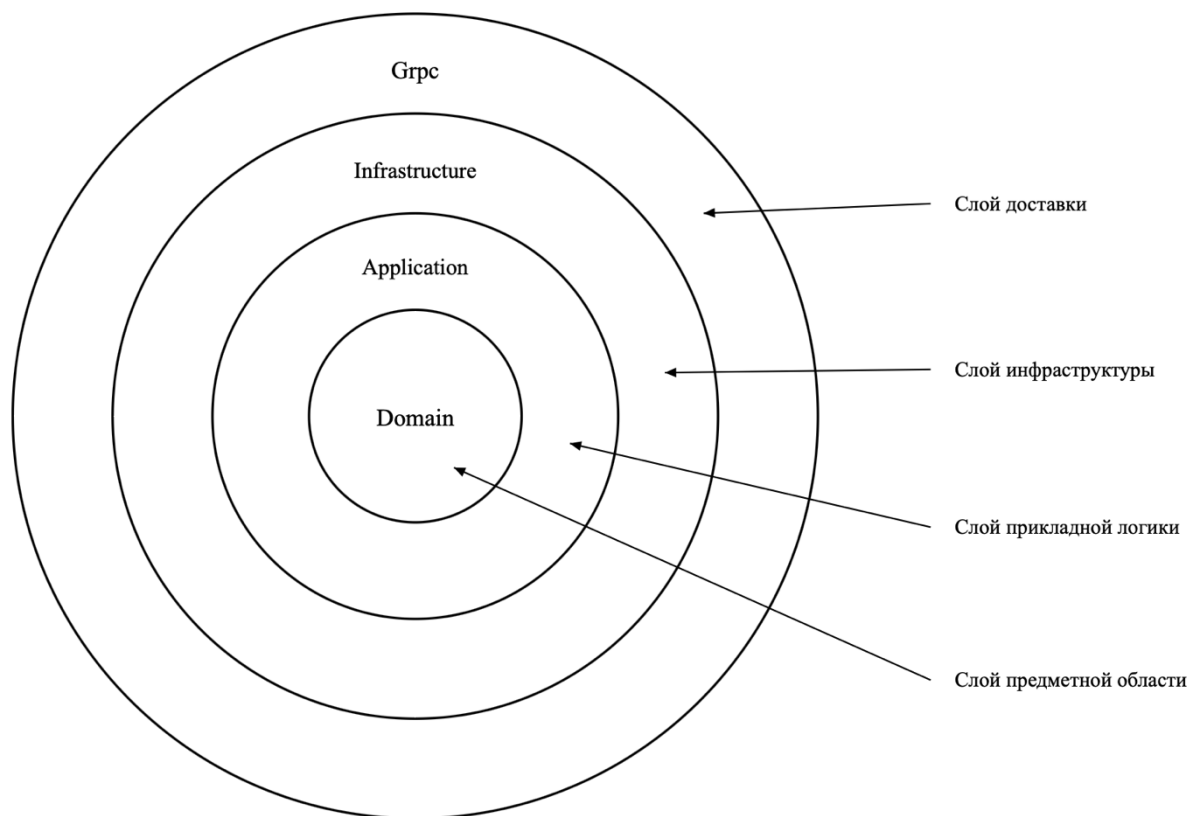


Рис. 2.1 Схема слоистой архитектуры платформы

2.2. Модель предметной области

Модель предметной области сосредоточена в слое предметной области и образует независимое ядро системы. Сущности модели наследуются от иерархии базовых типов, выражающей общие свойства. Базовый тип задает

единый идентификатор сущности в виде глобально уникального значения (GUID). Производный от него тип аудита добавляет отметки времени создания и последнего изменения, что обеспечивает прослеживаемость данных. Дальнейший производный тип вводит механизм «мягкого» удаления: вместо физического удаления записи помечаются признаком удаления и отметкой времени удаления, благодаря чему исторические данные пользователя не уничтожаются безвозвратно. Сущности персональных журналов наследуются именно от этого типа, а их «мягко» удаленные записи автоматически исключаются из выборок глобальным фильтром запросов.

Сущности модели делятся на три группы: справочный каталог, персональные связи пользователя с каталогом и персональные журналы.

Справочный каталог имеет иерархическую (древовидную) структуру. Корнем иерархии является диагноз. Каждому диагнозу соответствует множество лекарственных препаратов. Препарат описывается набором атрибутов – гормональная группа, международное непатентованное наименование, торговое наименование, дозировка, форма выпуска, частота приема и режим питания. Каждому препарату, в свою очередь, соответствует множество биологически активных добавок и множество побочных эффектов. Таким образом, каталог образует дерево «диагноз – препараты – добавки и побочные эффекты», в котором удаление вышестоящего узла каскадно затрагивает нижестоящие.

Персональные связи показывают, какие именно элементы каталога относятся к конкретному пользователю. Один пользователь может иметь несколько диагнозов, препаратов и добавок, и при этом один и тот же элемент каталога может относиться к разным пользователям. Чтобы выразить такую связь, используются промежуточные (связующие) сущности: каждая из них соединяет одного пользователя с одним элементом каталога. Назначенные диагнозы связывают пользователя с его диагнозами. Принимаемые препараты связывают пользователя с препаратами и дополнительно хранят даты начала и окончания приема и признак того, принимается ли препарат сейчас.

Принимаемые добавки устроены так же, но относятся к добавкам. Благодаря этому каждый пользователь получает собственную картину назначений, опираясь на общий каталог, и справочные данные при этом не дублируются.

Персональные журналы фиксируют динамику состояния пользователя во времени. Журнал побочных эффектов связывает запись с конкретным побочным эффектом каталога и хранит дату, интенсивность по шкале из четырех уровней (низкая, средняя, высокая, тяжелая) и комментарий. Журнал «внешних» препаратов хранит самостоятельно добавленные пользователем препараты, отсутствующие в каталоге, с наименованием, дозировкой, датой и комментарием, и не связан со справочными сущностями. Журнал записей менструального цикла хранит даты начала и окончания, интенсивность по шкале из четырех уровней (слабая, умеренная, обильная, очень обильная), перечень симптомов и заметки.

Перечисления модели задают замкнутые множества допустимых значений: роль пользователя, интенсивность побочного эффекта и интенсивность цикла. Хранение значений перечислений в виде строк повышает читаемость данных в базе и устойчивость к изменению порядка элементов.

Если рассматривать модель в терминах предметно-ориентированного проектирования [2], то пользователь играет роль «владельца» своих персональных данных: его связи с каталогом и журналы имеют смысл только применительно к нему и изменяются вместе с ним. Каталог же образует отдельную группу данных, главным элементом которой является диагноз. Правила, которые в модели должны выполняться всегда, поддерживаются на двух уровнях. На уровне проверки входных данных контролируется, например, что дата окончания приема позже даты начала, а интенсивность задана одним из допустимых значений. На уровне базы данных целостность обеспечивается связями между таблицами и правилами каскадного удаления, которые не позволяют появиться «осиротевшим» записям без владельца. Структура связей приведена на ER-диаграммах (см. Рис. 2.2.1-2.2.4).

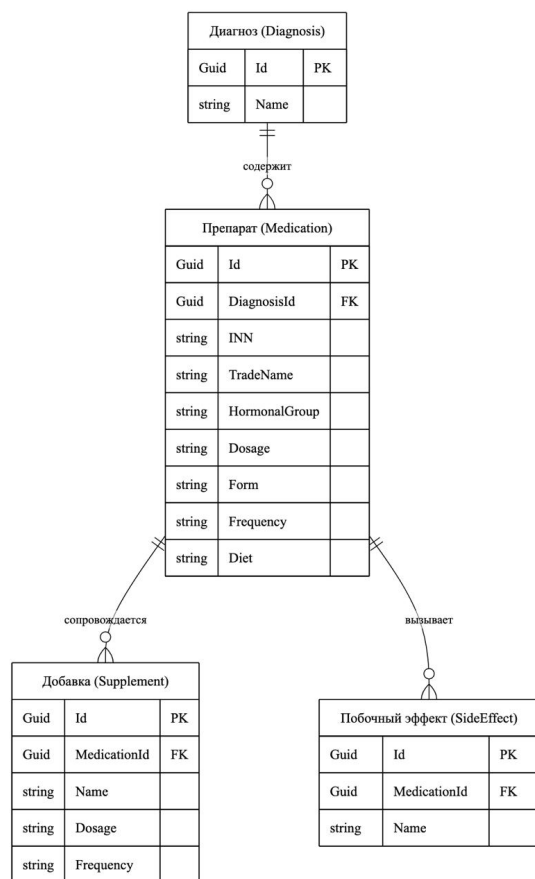


Рис. 2.2.1 ER-диаграмма справочного каталога

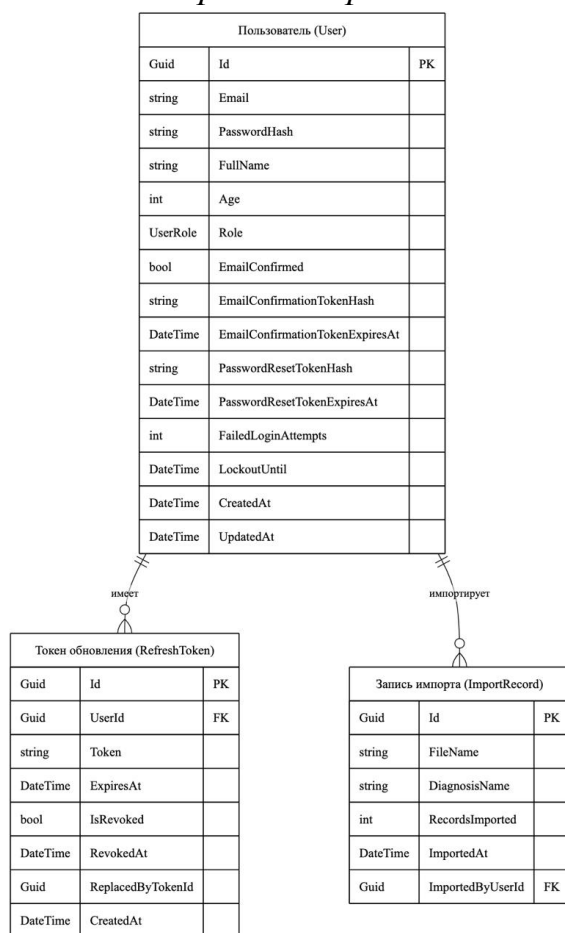


Рис. 2.2.2 ER-диаграмма учетной записи, безопасности и импорта

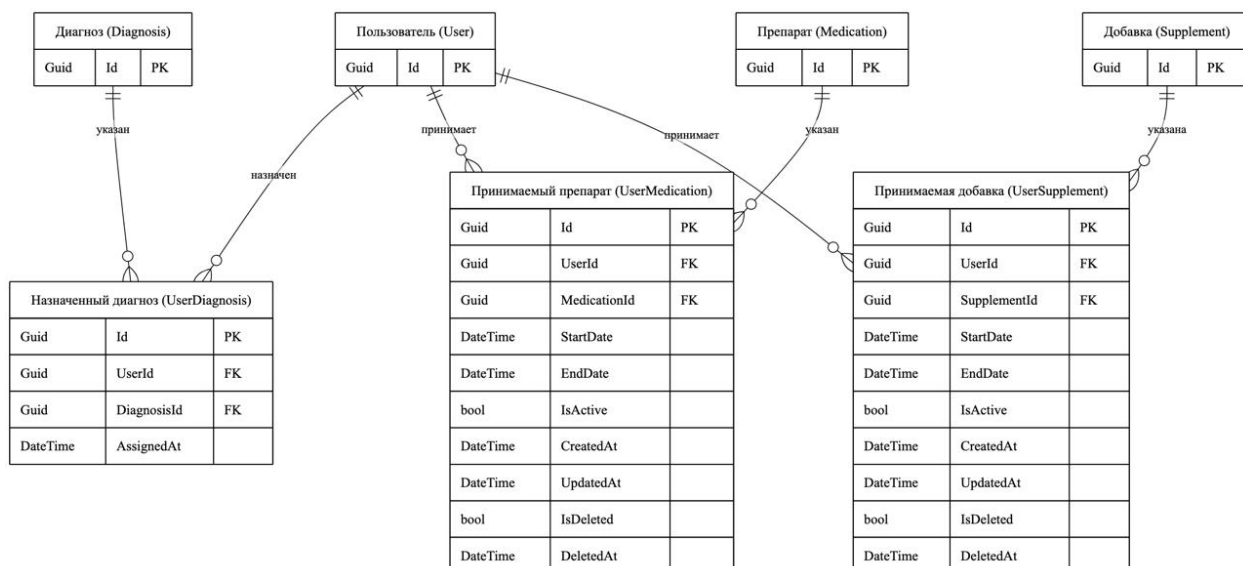


Рис. 2.2.3 ER-диаграмма назначений

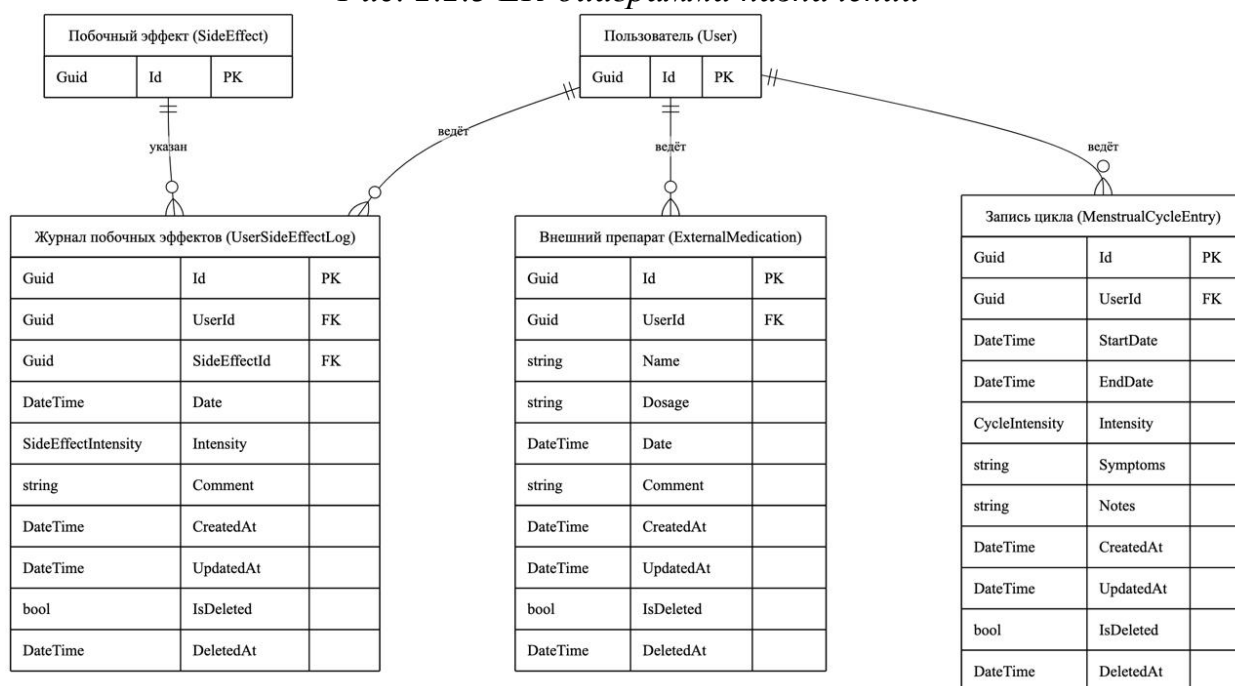


Рис. 2.2.4 ER-диаграмма журнала самонаблюдения

2.3. Проектирование программного интерфейса

Программный интерфейс платформы спроектирован как набор gRPC-сервисов, декомпозированных в соответствии с модульной структурой системы – по областям предметной области. Выделены восемь сервисов: сервис аутентификации, сервис профиля пользователя, сервис каталога препаратов, сервис назначений пользователя, сервис журнала побочных эффектов, сервис журнала внешних препаратов, сервис менструального цикла и административный сервис.

Контракты сервисов описаны в файлах Protocol Buffers, задающих как набор удаленных методов каждого сервиса, так и структуру сообщений запросов и ответов [11]. Контракт является единственным источником истины для клиента и сервера: на его основе автоматически генерируется код, что исключает рассогласование сторон [10, 17]. В контрактах используются стандартные типы-обертки: пустое сообщение для операций без параметров или без результата, метка времени для дат, типы-обертки для необязательных значений. Для операции массового импорта каталога применяется клиентский поток: административный сервис принимает файл по частям, что позволяет передавать крупные файлы без превышения лимита размера одиночного сообщения.

Выдача коллекций: каталога и журналов, спроектирована с постраничной навигацией. Запросы содержат параметры страницы и ее размера, а ответы – общее число элементов, что ограничивает объем передаваемых данных и нагрузку на хранилище. Запросы к журналам дополнительно поддерживают фильтрацию по диапазону дат.

Модель ошибок основана на стандартных кодах состояния gRPC. Доменные исключения и ошибки валидации не доходят до клиента в виде необработанных исключений: единый перехватчик исключений преобразует их в соответствующие коды состояния, нарушение правил валидации и некорректные аргументы отображаются в код «неверный аргумент», отсутствие запрошенной сущности в код «не найдено», конфликт уникальности в код «уже существует», отказ доступа в коды «не аутентифицирован» или «доступ запрещен», превышение лимита частоты в «ресурс исчерпан».

Чтобы можно было отследить путь отдельного запроса, каждому запросу присваивается уникальный идентификатор. Этот идентификатор сопровождает запрос на всем пути его обработки, записывается в логи и возвращается клиенту в ответе. Благодаря этому все записи в логах, относящиеся к одному запросу, можно собрать вместе по его идентификатору,

даже когда запросы обслуживаются несколькими одинаковыми экземплярами приложения и их логи перемешаны. Такой прием называется сквозной трассировкой.

2.4. Проектирование безопасности

Подсистема безопасности спроектирована исходя из того, что платформа обрабатывает специальную категорию персональных данных. Ее ядром является схема аутентификации на основе двух типов токенов. При регистрации и входе пользователю выдается пара: короткоживущий токен доступа в формате JSON Web Token [8] и долгоживущий токен обновления. Токен доступа предъявляется при каждом запросе и проверяется по подписи любой репликой без обращения к разделяемому хранилищу сеансов, что обеспечивает взаимозаменяемость реплик. Токен обновления служит для получения новой пары токенов без повторного ввода пароля и, в отличие от токена доступа, может быть отозван: смена пароля и выход аннулируют действующие токены обновления.

Часть методов сервиса аутентификации не требует предъявления токена – это методы регистрации, входа, обновления токена, подтверждения e-mail, повторной отправки подтверждения и запроса сброса пароля. Все прочие методы платформы доступны только аутентифицированным пользователям, а административные операции: импорт каталога и просмотр истории импортов только пользователям с ролью администратора. Тем самым реализована ролевая модель разграничения доступа с двумя ролями.

Подтверждение адреса электронной почты и сброс пароля спроектированы на одноразовых высокоэнтропийных токенах. В базе данных хранится не сам токен, а его хеш, вычисленный функцией SHA-256, а сравнение предъявленного значения с сохраненным выполняется в постоянном времени, что исключает утечку информации по времени сравнения. Пароли хранятся в виде хешей, вычисленных адаптивным алгоритмом BCrypt с настраиваемым параметром трудоемкости. Для защиты

от подбора пароля ведется счетчик неудачных попыток входа, где после пяти неудач учетная запись блокируется на 15 минут.

Для защиты от атак по времени отклика при обращении к несуществующей учетной записи выполняется проверка пароля против заранее заготовленного фиктивного хеша, благодаря чему время ответа не зависит от существования записи.

Каждый запрос, прежде чем дойти до бизнес-логики, проходит через цепочку обработчиков-перехватчиков, выстроенных в строгом порядке. Перехватчик – это промежуточный шаг, который что-то проверяет или добавляет к запросу и передает его дальше по цепочке.

Первым работает перехватчик идентификатора запроса: он берет идентификатор из заголовка запроса, а если его нет – создает новый, добавляет его в логи и возвращает клиенту. Вторым идет перехватчик ошибок: он перехватывает возникшие ошибки (нарушения бизнес-правил и неудачные проверки данных) и превращает их в понятные клиенту коды ответа gRPC. Третий – перехватчик аутентификации: он проверяет токен доступа, пропускает без проверки те методы, которым вход не требуется и убеждается, что пользователь не заблокирован, сверяясь с общим для всех экземпляров приложения кэшем. Четвертый – перехватчик ограничения частоты. Пятый – перехватчик размера запроса.

Запись идентификатора и обработка ошибок идут первыми, поэтому охватывают все последующие проверки. Проверка прав выполняется раньше тяжелых операций, а ограничения по частоте и размеру отсекают лишнюю нагрузку еще до того, как начнет работать бизнес-логика.

Последовательность взаимодействия при аутентификации – от входа и выдачи пары токенов до обновления токена доступа по токену обновления – представлена на схеме потока аутентификации (см. Рис. 2.4).

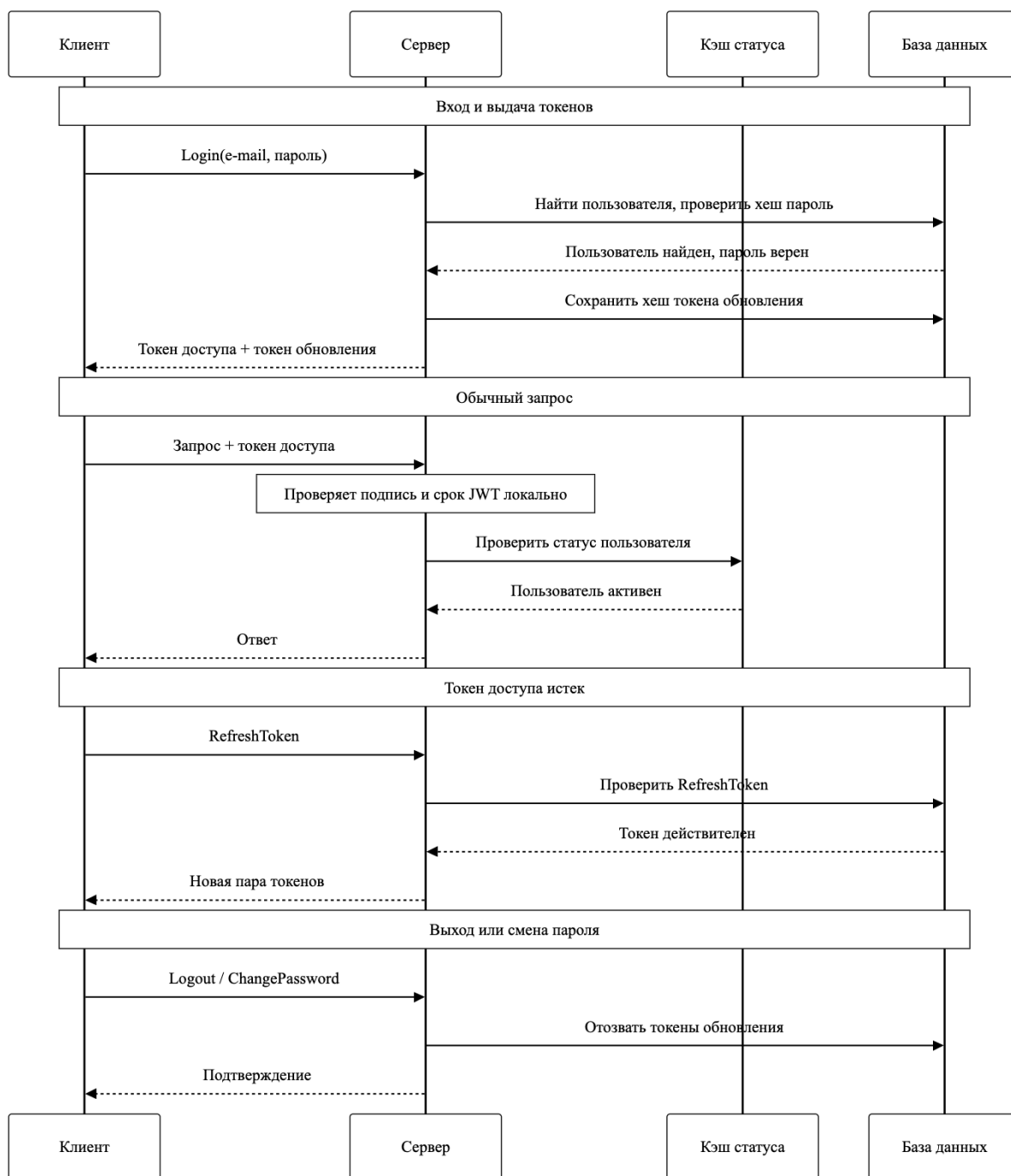


Рис. 2.4 Схема потока аутентификации

2.5. Проектирование обработки данных

Ряд операций платформы порождает необходимость внешнего действия – отправки письма с подтверждением адреса электронной почты или сбросом пароля. Как показано в разделе 1.4, выполнение фиксации данных и отправки письма вне единой транзакции порождает проблему двойной записи. Для ее устранения спроектирован транзакционный Outbox [5]. При выполнении

бизнес-операции сообщение о необходимости отправки письма записывается в специальную таблицу-«ящик исходящих» в той же транзакции базы данных, что и основная операция. Тем самым изменение данных и постановка письма в очередь фиксируются атомарно: либо сохраняется и то и другое, либо ничего. Само письмо в момент запроса не отправляется, поэтому временная недоступность почтового шлюза не нарушает основную операцию.

Доставку сообщений выполняет фоновый обработчик, периодически забирающий пакет накопленных сообщений и отправляющий их. Поскольку обработчик может исполняться при наличии нескольких реплик, спроектирована защита от одновременного захвата одного сообщения разными репликами. Захват пакета выполняется в транзакции с блокировкой выбранных строк и пропуском уже заблокированных, после чего захваченным сообщениям присваиваются токен блокировки и время его действия. Даже если реплика, захватившая пакет, аварийно завершится, сообщения останутся «защищенными» до истечения срока блокировки и не будут взяты другой репликой раньше времени, а по его истечении станут снова доступны для обработки.

Доставка сообщений организована по принципу «хотя бы один раз». При неуспешной отправке сообщение не теряется, а помечается для повторной попытки с растущей выдержкой: задержка перед очередной попыткой удваивается и составляет последовательно 30, 60, 120, 240 и 480 секунд. После исчерпания установленного числа попыток сообщение перестает забираться обработчиком, что предотвращает бесконечные повторы заведомо недоставляемых писем. Сбой при отправке одного сообщения не прерывает обработку остальных сообщений пакета. Для сквозной трассировки идентификатор корреляции исходного запроса сохраняется вместе с сообщением и при его обработке помещается в контекст логирования, благодаря чему путь «запрос → постановка письма → отправка → возможные повторы» прослеживается по единому идентификатору.

Планирование периодических и фоновых задач возложено на планировщик фоновых задач, использующий в качестве хранилища ту же реляционную базу данных. Спроектировано разделение ролей: реплики, обслуживающие программный интерфейс, работают в режиме «только постановка задач» – их сервер фоновых задач отключен, тогда как исполнение заданий и регистрация периодических задач возложены на выделенную рабочую реплику (worker). Такое разделение исключает дублирование исполнения периодических задач несколькими репликами и отделяет ресурсоемкую фоновую обработку от обслуживания запросов. В числе периодических задач: обработка пакета Outbox (выполняется ежеминутно), удаление давно обработанных сообщений Outbox, а также задачи сервиса очистки.

Сервис очистки спроектирован для поддержания базы данных в «опрятном» состоянии и удаления устаревших данных безопасности. Он периодически удаляет просроченные и отозванные токены обновления и обнуляет просроченные токены подтверждения адреса электронной почты и сброса пароля. Эти задачи выполняются по расписанию (ежесуточно в ночные часы), что снижает влияние на нагрузку в пиковые периоды.

2.6. Проектирование кэширования и ограничения нагрузки

При наличии нескольких реплик кэширование и ограничение нагрузки требуют согласованного, разделяемого между репликами состояния, поэтому несущим элементом этой подсистемы выбран распределенный кэш Redis [14].

Кэширование спроектировано как двухуровневое: первый уровень – локальный кэш в памяти отдельной реплики, обеспечивающий минимальную задержку, второй уровень – общий распределенный кэш в Redis, согласующий данные между репликами и переживающий их перезапуск. Двухуровневая схема применяется, в частности, для кэширования статуса пользователя, используемого перехватчиком аутентификации (с коротким временем жизни на первом уровне и более длительным – на втором), и для кэширования

справочного каталога. При обращении к статусу пользователя реплика сначала проверяет локальный кэш, затем распределенный и лишь в последнюю очередь базу данных, что снижает нагрузку на хранилище при частых проверках.

Согласованность кэша каталога обеспечивается версионированием. Во внешнем разделяемом хранилище ведется атомарный счетчик версии каталога, значение которого включается в ключи кэша. При любом изменении каталога, прежде всего при административном импорте, счетчик версии атомарно увеличивается. После этого все реплики при построении ключа кэша используют новое значение версии, поэтому записи, закешированные под прежней версией, перестают использоваться и со временем вытесняются по истечении срока жизни. Такая «мягкая» инвалидация не требует одновременного оповещения всех реплик и согласованно работает в распределенной среде [4]: каждая реплика самостоятельно переходит на актуальные данные, сверяясь со счетчиком версии.

Ограничение частоты запросов спроектировано как распределенное, чтобы лимит соблюдался суммарно по всем репликам, а не по каждой в отдельности. Подсчет обращений выполняется в Redis атомарными скриптами, исключаяющими «гонки» при одновременных запросах с разных реплик. Для большинства методов применяется счетчик в фиксированном окне (атомарное увеличение значения с установкой срока жизни ключа при первом обращении), а для наиболее чувствительных к злоупотреблениям методов: входа, регистрации и обновления токена – более точная схема скользящего окна. Подсистема ограничения нагрузки спроектирована по политике «отказ в сторону доступности»: если распределенное хранилище временно недоступно, запросы не блокируются, а пропускаются, чтобы недоступность вспомогательной службы не приводила к отказу в обслуживании законных пользователей. Факт недоступности при этом фиксируется в логах.

Наконец, в том же распределенном хранилище размещены ключи подсистемы защиты данных, общие для всех реплик. Это необходимо для того, чтобы данные, защищенные одной репликой, могли быть корректно обработаны любой другой репликой. Без общего набора ключей взаимозаменяемость реплик нарушилась бы. Тем самым все элементы разделяемого состояния: кэш, счетчик версии каталога, счетчики ограничения частоты и ключи защиты данных – вынесены во внешнее хранилище, что обеспечивает взаимозаменяемость реплик.

2.7. Проектирование хранения данных

Данные хранятся в реляционной базе данных PostgreSQL [13]. Она была выбрана так как, в системе много связей между сущностями, и реляционная база позволяет описать эти связи и сама следит за их корректностью. Например, не дает сослаться на несуществующую запись. Приложение обращается к базе не напрямую, а через средство объектно-реляционного отображения – Entity Framework Core с провайдером Npgsql [16]. Оно автоматически переводит объекты в строки таблиц и обратно. Поверх него работает слой репозитория – набор классов, который собирает в одном месте все операции чтения и записи и скрывает от прикладной логики, как именно строятся запросы [6].

Соответствие между объектами модели и таблицами базы задается в отдельных классах-настройках для каждой сущности. В них указано, какого типа и с какими ограничениями каждое поле, как связаны таблицы (внешние ключи) и что при удалении узла каталога должны удаляться и подчиненные ему записи (каскадное удаление в дереве «диагноз – препараты – добавки и побочные эффекты»). Значения перечислений хранятся в виде строк, чтобы данные в базе было легче читать. Кроме того, настроен общий фильтр, который автоматически убирает из всех выборок записи журналов, помеченные как удаленные, поэтому такие записи не приходится отдельно отсеивать в каждом запросе.

В распределенной среде с несколькими репликами особое значение приобретает управление соединениями с базой данных. Каждая реплика, открывая собственные соединения, в совокупности способна исчерпать предельное число соединений сервера базы данных. Для решения этой проблемы между приложением и базой данных размещен пул соединений PgBouncer, уплотняющий большое число клиентских соединений в ограниченное число серверных. Через пул соединений настроены два логических подключения к одной и той же базе данных, работающих в разных режимах. Основное подключение работает в режиме пула на уровне транзакций: серверное соединение выдается клиенту лишь на время одной транзакции, а затем сразу возвращается в пул и достается следующему запросу. Так одно физическое соединение успевает обслужить множество запросов, что подходит для обычного трафика приложения, где операции короткие. Отдельное подключение работает в режиме пула на уровне сеанса: здесь серверное соединение закрепляется за клиентом на все время его работы и не передается другим. Такой режим нужен планировщику фоновых задач и инструменту применения миграций – им требуется устойчивое соединение, которое нельзя переключать в середине работы.

Выдача коллекций каталога и журналов спроектирована постранично, что ограничивает объем данных, читаемых и передаваемых за один запрос. Отдельного внимания требует стратегия применения миграций схемы базы данных. Применение миграций спроектировано как отдельный шаг развертывания, а не как действие, выполняемое самим приложением при старте. Если бы каждая реплика применяла миграции при запуске, то при одновременном обновлении нескольких реплик возникла бы гонка за изменение схемы. Поэтому миграции применяются однократно выделенным заданием перед обновлением реплик приложения, что исключает состояние гонки при поэтапном обновлении и гарантирует, что все реплики работают с согласованной версией схемы.

2.8. Проектирование развертывания и масштабирования

Развертывание платформы спроектировано на основе контейнеризации: приложение упаковывается в контейнерный образ со всеми зависимостями, что обеспечивает единообразное исполнение в разных средах. Вспомогательная инфраструктура для локальной разработки – база данных, пул соединений и распределенный кэш описывается в одном конфигурационном файле, где перечислены все нужные контейнеры. Это позволяет запускать ее целиком одной командой. Для эксплуатации и масштабирования применяется оркестратор Kubernetes [12]. Конфигурация системы для Kubernetes задается набором манифестов – файлов, описывающих желаемое состояние. Чтобы не дублировать описания для разных сред, применяется прием «общая база плюс надстройка»: есть базовый набор манифестов и отдельная надстройка с отличиями для среды разработки, которая накладывается поверх базы специальным инструментом. Все ресурсы системы размещены в отдельном пространстве имен – изолированной области внутри кластера, чтобы они не смешивались с другими приложениями.

Входящий трафик попадает к репликам через входной компонент маршрутизации, который умеет работать и с gRPC, и с обычным HTTP. По пути он добавляет в стандартные заголовки запроса настоящий адрес клиента.

Горизонтальное автомасштабирование прикладной части спроектировано на основе механизма автомасштабирования по показателям нагрузки [12]. Число реплик прикладного интерфейса изменяется автоматически в диапазоне от двух до десяти: нижняя граница в две реплики поддерживает доступность даже при отсутствии нагрузки, верхняя в десять реплик ограничивает рост потребления ресурсов. Целевыми показателями выбраны средняя загрузка процессора 70 % и средняя загрузка памяти 90 %. При их превышении число реплик увеличивается, при снижении – уменьшается. Поведение масштабирования спроектировано асимметрично: увеличение числа реплик происходит быстро (с окном реакции 15 секунд), поскольку предпочтительнее иметь избыточную реплику, чем допустить рост задержек, тогда как

уменьшение выполняется осторожно, после окна стабилизации в пять минут, что предотвращает «хлопанье» реплик при кратковременных скачках нагрузки. Рабочая реплика, исполняющая фоновые задачи, через механизм автомасштабирования не масштабируется: планировщик с распределенными блокировками работает с одной активной репликой, а дополнительные реплики служат лишь страховкой от отказа и не увеличивают пропускную способность.

Обновление платформы спроектировано без простоя обслуживания. Применяется стратегия поэтапного обновления с запретом одновременного снижения числа доступных реплик ниже заданного: новые реплики запускаются и проходят проверку готовности прежде, чем выводятся из эксплуатации старые. Готовность и работоспособность реплик определяются соответствующими пробами, а для каждой реплики заданы запросы и лимиты вычислительных ресурсов, что необходимо как для корректной работы автомасштабирования, так и для предсказуемого размещения реплик по узлам. На всем пути обработки запроса ведется структурированное логирование с единым идентификатором запроса, по которому можно собрать вместе все относящиеся к нему записи в логах. Общая схема развертывания платформы – реплики приложения, число которых меняется автоматически, отдельная рабочая реплика для фоновых задач, распределенный кэш, пул соединений и база данных, а также маршрутизация входящего трафика показана на рисунке 2.8.

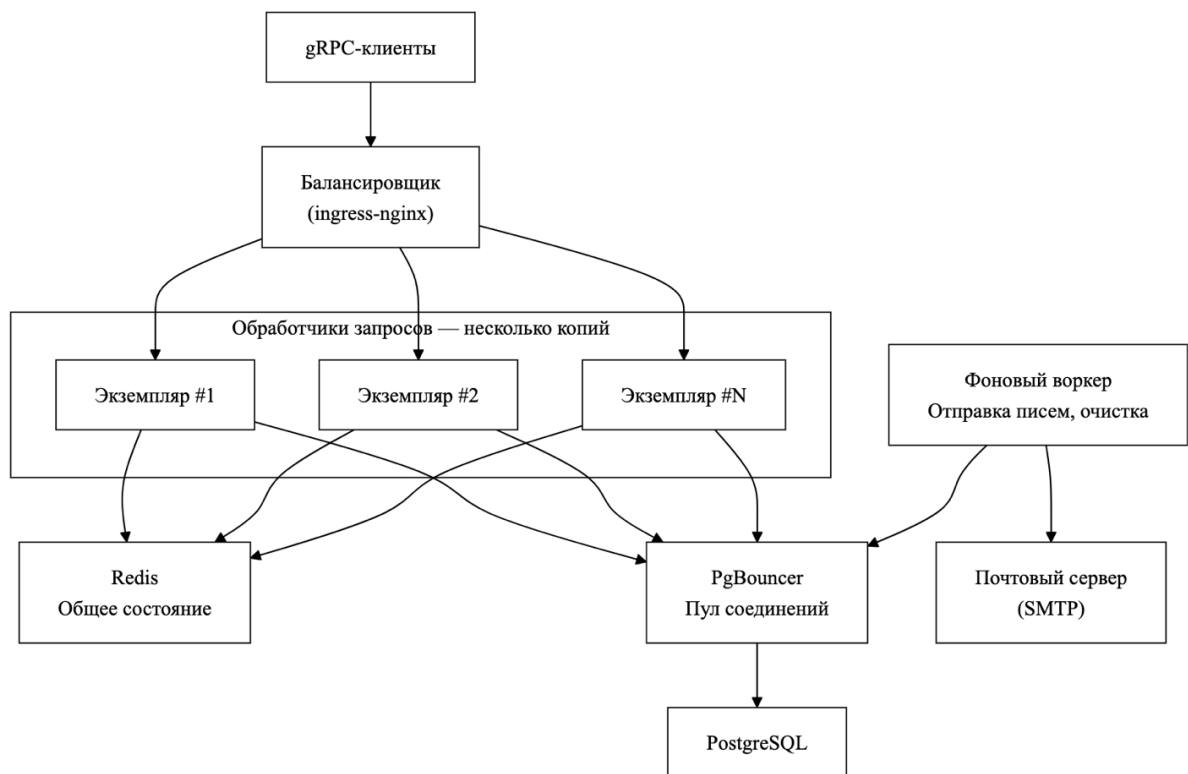


Рис. 2.8. Схема развертывания платформы

3. ПРАКТИЧЕСКАЯ РЕАЛИЗАЦИЯ

3.1. Организация программного решения

Решение MedTracker реализовано на .NET 10 на языке C# [15] и объединено файлом MedTracker.slnx. Оно разделено на четыре проекта-слоя в соответствии со слоистой архитектурой. Слой предметной области (проект MedTracker.Domain) хранит сущности и не зависит ни от чего внешнего: иерархия базовых типов задана в файле BaseEntity, перечисления в Enums, доменные исключения в DomainExceptions. Слой прикладной логики (проект MedTracker.Application) содержит интерфейсы сервисов, их реализации, объекты передачи данных в файле Dtos и правила проверки в файле Validators. Слой инфраструктуры (проект MedTracker.Infrastructure) реализует объявленные прикладным слоем интерфейсы – доступ к данным, кэш, почту, фоновые задачи. Слой доставки (проект MedTracker.Grpc) – точка входа: он поднимает gRPC-сервер и цепочку перехватчиков.

Регистрация сервисов и валидаторов прикладного слоя сосредоточена в файле DependencyInjection: его метод AddApplication добавляет в контейнер реализации интерфейсов IAuthService, IMedicationCatalogService, IUserMedicationService и прочих, а также подключает валидаторы FluentValidation одним вызовом. Окончательная сборка – сопоставление каждому интерфейсу его инфраструктурной реализации выполняется в файле Program слоя доставки, который служит точкой сборки приложения. Компоненты получают зависимости через конструкторы, поэтому ни один сервис не создает свои зависимости сам. Благодаря такому связыванию реализации зависят от абстракций прикладного слоя, а не наоборот, и заменяются без правки бизнес-логики.

3.2. Реализация слоя данных

Слой данных реализован на Entity Framework Core с провайдером Npgsql поверх PostgreSQL. Центральный класс – контекст AppDbContext: он

объявляет наборы сущностей и в методе настройки модели применяет все конфигурации, а также устанавливает глобальный фильтр запросов, который автоматически отбрасывает «мягко» удаленные записи журналов из любых выборок.

Соответствие сущностей таблицам вынесено из самих сущностей в отдельные файлы конфигураций. Каталог, связи пользователя и журналы настроены в файле EntityConfigurations: там для древовидного каталога заданы связи «один ко многим» с каскадным удалением, а значения перечислений преобразуются в строки. Учетная запись, токен обновления и сообщения Outbox настроены в файле UserAndOutboxConfigurations, где, в частности, объявлен уникальный индекс по адресу электронной почты, обеспечивающий уникальность учетных записей и быстрый поиск при входе.

Доступ к данным организован репозиториями: их интерфейсы объявлены в файле IRepositories, а реализации в файле Repositories. Каждый репозиторий предоставляет операции выборки и сохранения, а страничные запросы возвращают тип PaginatedResultDto, содержащий страницу элементов и общее число записей. Наиболее содержательна реализация репозитория Outbox в файле OutboxRepository, рассчитанная на работу нескольких реплик. Его метод ClaimBatchAsync выбирает готовые к отправке сообщения запросом с конструкцией FOR UPDATE SKIP LOCKED – он блокирует выбранные строки и пропускает уже заблокированные другими репликами и в той же транзакции присваивает захваченным сообщениям токен и время блокировки (поля LockToken и LockedUntil). Методы MarkProcessedAsync и MarkFailedAsync помечают сообщение обработанным либо назначают время следующей попытки, а константа MaxRetryCount, равная пяти, исключает из выборки сообщения, исчерпавшие попытки.

3.3. Реализация прикладной логики

Прикладная логика сосредоточена в сервисах слоя MedTracker.Application. gRPC-сервисы слоя доставки лишь вызывают их. Каждый сервис реализует свой интерфейс из прикладного слоя.

Центральный сервис аутентификации реализован в файле AuthService и закрывает весь жизненный цикл учетной записи. Метод RegisterAsync проверяет отсутствие пользователя с таким адресом, хеширует пароль через BCrypt и создает учетную запись с неподтвержденным адресом, а письмо с подтверждением не отправляет напрямую, а ставит в очередь через репозиторий Outbox в одной транзакции с созданием пользователя. Метод LoginAsync реализует вход с несколькими защитными проверками: сначала проверяется блокировка учетной записи. Пароль сверяется методом Verify хешировщика, а при отсутствии пользователя – против заранее заданной константы DummyHash, чтобы время ответа не выдавало существование записи. При неверном пароле увеличивается поле FailedLoginAttempts, и по достижении константы MaxFailedAttempts, равной пяти, выставляется LockoutUntil на пятнадцать минут. Существенно, что подтверждение адреса проверяется только после успешной сверки пароля – это исключает определение по разным ответам, существует ли и подтвержден ли аккаунт. При успешном входе счетчик и блокировка сбрасываются, после чего вызывается приватный метод GenerateTokensAsync, выдающий пару токенов. Метод RefreshTokenAsync, помимо проверки срока действия, реализует защиту от повторного использования: если предъявлен уже отозванный токен, это трактуется как признак компрометации, и метод RevokeAllForUserAsync отзывает все токены обновления пользователя, а в лог пишется предупреждение. Сервис каталога в файле MedicationCatalogService реализует методы GetDiagnosesAsync, GetMedicationsByDiagnosisAsync и подобные, читая данные через двухуровневый кэш по ключу, включающему версию каталога. Сервисы персональных данных – UserProfileService и UserMedicationService реализуют операции вроде AssignMedicationAsync и

RemoveMedicationAsync, причем идентификатор пользователя они получают параметром из контекста запроса, поэтому операции всегда выполняются в границах конкретного пользователя. Сервисы журналов – SideEffectLogService, ExternalMedicationService и MenstrualCycleService реализуют создание, удаление и постраничную выдачу с фильтрацией по диапазону дат (например, метод GetLogsAsync принимает границы периода, номер и размер страницы), также в границах пользователя.

Проверка входных данных вынесена в валидаторы файла Validators на основе FluentValidation и выполняется до бизнес-логики. Проверяются формат адреса электронной почты, требования к паролю, допустимый возраст, обязательность полей и согласованность дат. Нарушения валидации не доходят до бизнес-логики, а преобразуются в код ошибки на уровне перехватчика.

3.4. Реализация API и перехватчиков

gRPC-сервисы реализованы как тонкие адаптеры между транспортом и прикладной логикой. Они размещены в файлах вида AuthGrpcService, UserMedicationGrpcService, MedicationCatalogGrpcService и так далее. Типовой метод устроен одинаково: он извлекает идентификатор пользователя из контекста вызова вспомогательным методом GetUserId, преобразует входное сообщение Protocol Buffers в объект передачи данных, вызывает соответствующий сервис прикладного слоя и преобразует результат обратно в сообщение-ответ. Например, метод AssignMedication в файле UserMedicationGrpcService собирает объект AssignMedicationDto из полей запроса и вызывает AssignMedicationAsync прикладного сервиса. Бизнес-логику gRPC-сервисы не содержат, что сохраняет разделение слоев.

Перехватчик из файла CorrelationIdInterceptor считывает идентификатор из заголовка x-correlation-id либо генерирует новый, помещает его в контекст логирования и возвращает клиенту в trailer-заголовках. Доступ к этому идентификатору из прикладного кода обеспечивает реализация интерфейса

ICorrelationIdAccessor в файле CorrelationIdAccessor, что позволяет сохранять идентификатор в сообщениях Outbox. Перехватчик из файла ExceptionInterceptor оборачивает выполнение и преобразует доменные исключения из файла DomainExceptions и ошибки валидации в коды состояния gRPC: «не найдено» в NotFound, нарушение прав в Unauthenticated или PermissionDenied, конфликт уникальности в AlreadyExists, ошибки валидации в InvalidArgument, а необработанные исключения во внутреннюю ошибку без раскрытия деталей.

Перехватчик из файла AuthInterceptor реализует контроль доступа. Для методов из списка анонимных (регистрация, вход, обновление токена, подтверждение и повторная отправка подтверждения, запрос сброса пароля) проверка пропускается. Для остальных методов проверяется токен доступа, идентификатор и роль пользователя помещаются в контекст вызова, а статус пользователя (существование и отсутствие блокировки) сверяется с двухуровневым кэшем через реализацию интерфейса IUserStatusCache, что позволяет немедленно отклонять запросы заблокированных учетных записей без обращения к базе при каждом запросе. Перехватчик из файла RateLimitInterceptor формирует ключ из реального адреса клиента и имени метода и обращается к ограничителю IRateLimiter. При превышении лимита возвращается код ResourceExhausted. Перехватчик из файла RequestSizeInterceptor отклоняет сообщения сверх установленного предела, с увеличенным лимитом для импорта каталога.

3.5. Реализация безопасности

Подсистема безопасности опирается на несколько файлов слоя инфраструктуры. Выдача и устройство токенов сосредоточены в файле JwtService. Его метод GenerateAccessToken формирует набор утверждений (claims) – идентификатор пользователя, адрес электронной почты, роль, уникальный идентификатор токена и подписывает токен алгоритмом HMAC-SHA256 секретом из конфигурации. Срок жизни токена доступа берется из

настройки и по умолчанию составляет пятнадцать минут. Метод `GenerateRefreshToken` создает токен обновления из 64 случайных байт, а сам токен сохраняется со сроком действия семь суток. Хеширование паролей реализовано отдельным компонентом, реализующим интерфейс `IPasswordHasher` с методами `Hash` и `Verify` поверх алгоритма `BCrypt`.

Одноразовые токены подтверждения адреса и сброса пароля реализованы в файле `TokenGenerator`. Его метод `GenerateToken` создает 32-байтовое случайное значение и возвращает пару: открытый токен в кодировке `Base64Url` (он отправляется пользователю в письме и безопасно вставляется в URL) и его хеш `SHA-256` (он сохраняется в базе данных). Метод `Verify` повторно вычисляет хеш предъявленного токена и сравнивает его с сохраненным в постоянном времени вызовом `FixedTimeEquals`, что исключает утечку информации по времени сравнения. Тем самым в базе данных никогда не хранится сам токен, только его хеш.

Защитные проверки при входе реализованы в сервисе аутентификации. Дополнительно реализовано аннулирование сессий при критических действиях: методы смены и сброса пароля вызывают `RevokeAllForUserAsync`, отзывающий все токены обновления пользователя. Контроль доступа при каждом запросе обеспечивает перехватчик аутентификации совместно с кэшем статуса пользователя, а ролевое разграничение (из перечисления `UserRole`) ограничивает административные операции пользователями с ролью администратора.

3.6. Реализация доставки и фоновых задач

Надежная доставка служебных писем реализована по схеме транзакционного Outbox. Сервис аутентификации не отправляет письма напрямую: его вспомогательный метод `EnqueueConfirmationEmailAsync` формирует адрес перехода через построитель ссылок (файл `AppUrlBuilder`), рендерит шаблон письма через файл `EmailTemplateService` и добавляет сообщение в очередь методом `AddAsync` репозитория Outbox в той же

транзакции, что и бизнес-операция. При этом в сообщение записывается текущий идентификатор корреляции.

Доставку выполняет обработчик из файла `OutboxJob`. Его метод `ProcessBatchAsync` захватывает пакет сообщений методом `ClaimBatchAsync` репозитория, затем для каждого сообщения отправляет письмо через реализацию интерфейса `IMailSender` и помечает сообщение обработанным методом `MarkProcessedAsync`. При сбое отправки вызывается `MarkFailedAsync`, назначающий время следующей попытки с растущей выдержкой. Сбой одного сообщения не прерывает обработку остальных в пакете. Для сквозной трассировки обработчик помещает идентификатор корреляции исходного запроса в контекст логирования через конструкцию `LogContext`, поэтому записи об отправке письма содержат тот же идентификатор, что и исходный запрос. Параметры обработчика (размер пакета, длительность блокировки, базовая задержка, срок хранения обработанных сообщений) вынесены в файл `OutboxOptions`.

Планирование периодических задач реализовано на планировщике `Hangfire` [19]. Регистрация заданий выполнена в файле `RecurringJobsRegistrar`, реализующем интерфейс службы: его метод `StartAsync` регистрирует ежеминутную обработку пакета `Outbox`, ежечасную очистку давно обработанных сообщений и ежесуточные задачи очистки просроченных токенов. Сами задачи очистки реализованы в файле `CleanupService` методами `CleanupExpiredRefreshTokensAsync`, `CleanupExpiredEmailConfirmationTokensAsync` и `CleanupExpiredPasswordResetTokensAsync`, удаляющими просроченные и отозванные токены обновления и обнуляющими просроченные одноразовые токены.

Наполнение каталога реализовано в файле `ExcelImportService`: административный сервис принимает файл по частям, разбирает табличные данные и сохраняет справочные записи, фиксируя факт загрузки записью истории импорта. По завершении импорта каталог считается измененным, и

версия кэша каталога увеличивается, чтобы реплики перешли на актуальные данные.

3.7. Реализация кэширования и ограничения нагрузки

Кэширование реализовано как двухуровневое средствами HybridCache (первый уровень – в памяти реплики, второй – в Redis). Кэш каталога реализован в сервисе MedicationCatalogService: методы вида GetDiagnosesAsync и GetMedicationsByDiagnosisAsync читают данные через метод GetOrCreateAsync кэша, причем ключ кэша включает текущую версию каталога и имеет вид «catalog:...». Записи каталога живут в кэше до часа, а саму версию сервис тоже кэширует методом GetVersionAsync с коротким локальным сроком жизни (15 секунд) и более длительным распределенным (5 минут).

Версия каталога хранится в файле CatalogVersionStore под ключом medtracker:catalog:version. Метод GetCurrentAsync возвращает текущую версию каталога. Если счетчика версии в Redis еще нет, метод создает его при первом обращении со значением 1, причем запись выполняется только при условии, что счетчика еще не существует, это исключает ошибки при одновременном обращении с разных реплик. Метод BumpAsync увеличивает версию на единицу операцией Redis INCR, которая выполняется атомарно, поэтому одновременные вызовы не мешают друг другу. Инвалидация реализована в классе CatalogCacheInvalidator: его метод InvalidateAsync вызывается после импорта каталога и просто увеличивает версию. После этого локальный кэш версии во всех репликах истекает максимум за пятнадцать секунд, реплики начинают строить ключи с новой версией, а записи под прежней версией перестают читаться и истекают по своему сроку жизни. Тем самым реализована «мягкая» инвалидация, не требующая одновременного оповещения всех реплик.

Кэш статуса пользователя реализован в файле UserStatusCache: его метод GetAsync возвращает снимок UserStatusSnapshot, содержащий признак

существования учетной записи и время ее блокировки, читая его через двухуровневый кэш. Метод `InvalidateAsync` принудительно сбрасывает кэш, например после смены пароля или выхода. Этот кэш использует перехватчик аутентификации, чтобы не обращаться к базе при каждом запросе.

Распределенное ограничение частоты реализовано в файле `RedisRateLimiter`. Его метод `CheckAsync` выполняет в Redis атомарный сценарий, что исключает гонки при одновременных запросах с разных реплик. Метод возвращает результат типа `RateLimitResult` с признаком разрешения, текущим значением и временем сброса.

3.8. Контейнеризация и оркестрация

Развертывание реализовано набором файлов контейнеризации и манифестов. Сборка образа приложения описана в файле `Dockerfile` многоэтапной сборкой, а локальная вспомогательная инфраструктура (база данных, пул соединений, распределенный кэш) поднимается файлом `compose.yaml`. Пул соединений настроен файлом `Pgbouncer.ini`, в котором заданы два логических подключения к базе в разных режимах пула.

Развертывание в кластере описано декларативными манифестами Kubernetes: пространство имен (`namespace.yaml`), развертывания прикладного интерфейса и рабочей реплики (`api-deployment.yaml` и `worker-deployment.yaml`), служба (`api-service.yaml`), входной маршрутизатор (`ingress.yaml`), конфигурация и секреты (`configmap.yaml`, `secret.yaml`). Манифесты организованы средствами `kustomize`, что задает файл `kustomization.yaml`. Применение миграций вынесено в отдельное задание (`migrate-job.yaml`), выполняемое до обновления реплик. Для локального кластера предусмотрены файл `kind-config.yaml` и заплатка к серверу метрик (`metrics-server-patch.yaml`), а для удобства – сценарии `k8s-up.sh` и `k8s-down.sh`. Горизонтальное автомасштабирование описано файлом `hpa.yaml`.

3.9. Тестирование

Корректность реализации проверяется двумя проектами тестов. Модульные тесты собраны в проекте MedTracker.Tests с использованием xUnit, библиотеки подстановок NSubstitute и библиотеки утверждений FluentAssertions. Интеграционные в проекте MedTracker.IntegrationTests, работающем с настоящими PostgreSQL и Redis.

Модульные тесты покрывают компоненты с нетривиальной логикой. Файл AuthServiceTests проверяет ключевые сценарии сервиса аутентификации: при регистрации создается пользователь и письмо ставится в очередь Outbox. Файл OutboxJobTests подтверждает рост выдержки повторов и то, что сбой одного сообщения не прерывает обработку остальных. Файл MedicationCatalogServiceTests проверяет логику кэширования: первый запрос идет в репозиторий, повторные берутся из кэша, а после увеличения версии данные читаются заново. Дополнительно проверяются генератор токенов TokenGeneratorTests, инвалидатор кэша CatalogCacheInvalidatorTests и перехватчик ограничения частоты RateLimitInterceptorTests. Для удобства реализованы вспомогательные средства: построитель тестового пользователя UserBuilder, набор подготовки сервиса аутентификации AuthServiceTestSetup и подставной контекст вызова TestServerCallContext.

Интеграционные тесты используют общий стенд из файла IntegrationTestsFixtures, который поднимается один раз на все тесты и между тестами очищает базу данных (усечением таблиц) и Redis. Файл OutboxRepositoryTests проверяет безопасный захват пакета через «заблокировать или пропустить», установку выдержки при неудаче и исключение из выборки сообщений, исчерпавших попытки. Файл CatalogCacheIntegrationTests на настоящем Redis проверяет полный цикл «чтение – кэширование – инвалидация – повторное чтение»: после увеличения версии запрос возвращает обновленные данные, а отдельный тест подтверждает атомарность увеличения счетчика при пятидесяти одновременных операциях. Файлы RedisRateLimiterTests и

CleanupServiceTests проверяют распределенное ограничение частоты и удаление просроченных и отозванных токенов с сохранением действующих.

Масштабируемость проверяется нагрузочным тестированием по сценарию из файла load-test.sh. Сценарий запускает заданное число параллельных потоков, каждый из которых в цикле выполняет запрос входа. Поскольку вход выполняет ресурсоемкое хеширование пароля BCrypt, нагрузка приходится на процессор – именно тот показатель, по которому настроено автомасштабирование. Методика состоит в следующем: при наблюдении за состоянием автомасштабирования и реплик средствами оркестратора создается устойчивая нагрузка, под которой средняя загрузка процессора превышает целевые 70 %. Ожидаемым результатом является автоматическое увеличение числа реплик прикладного интерфейса вплоть до десяти, а после снятия нагрузки и истечения окна стабилизации – обратное уменьшение до двух. Тем самым подтверждается работоспособность горизонтального автомасштабирования. Результаты теста нагрузки будут предоставлены на рисунках 3.9.1.-3.9.3.

Every 2.0s: kubectl get hpa,pods -n medtracker

MacBook-Pro-Vlad.local: Wed Jun 10 16:09:56 2026
in 0.078s (0)

NAME	REFERENCE	TARGETS	MINPODS	MAXPODS	REPLICAS	AGE
horizontalpodautoscaler.autoscaling/medtracker-api	Deployment/medtracker-api	cpu: 14%/70%, memory: 45%/90%	2	10	3	79s

NAME	READY	STATUS	RESTARTS	AGE
pod/medtracker-api-54dbcb95-2mccq	1/1	Running	0	79s
pod/medtracker-api-54dbcb95-fk2m4	1/1	Running	0	79s
pod/medtracker-api-54dbcb95-rp475	1/1	Running	0	79s
pod/medtracker-migrate-jbfxr	0/1	Completed	0	79s
pod/medtracker-worker-54b6869fb9-hlzp	1/1	Running	0	79s

Рис 3.9.1. Состояние системы до нагрузочного теста

Every 2.0s: kubectl get hpa,pods -n medtracker

MacBook-Pro-Vlad.local: Wed Jun 10 16:11:19 2026
in 0.333s (0)

NAME	REFERENCE	TARGETS	MINPODS	MAXPODS	REPLICAS	AGE
horizontalpodautoscaler.autoscaling/medtracker-api	Deployment/medtracker-api	cpu: 777%/70%, memory: 44%/90%	2	10	10	2m42s

NAME	READY	STATUS	RESTARTS	AGE
pod/medtracker-api-54dbcb95-2mccq	1/1	Running	0	2m42s
pod/medtracker-api-54dbcb95-88zm4	1/1	Running	0	42s
pod/medtracker-api-54dbcb95-9dpjc	1/1	Running	0	42s
pod/medtracker-api-54dbcb95-cmbgr	1/1	Running	0	42s
pod/medtracker-api-54dbcb95-d7khl	1/1	Running	0	27s
pod/medtracker-api-54dbcb95-fk2m4	1/1	Running	0	2m42s
pod/medtracker-api-54dbcb95-r2zp2	1/1	Running	0	27s
pod/medtracker-api-54dbcb95-rp475	1/1	Running	0	2m42s
pod/medtracker-api-54dbcb95-tdj4l	1/1	Running	0	42s
pod/medtracker-api-54dbcb95-xxf62	1/1	Running	0	27s
pod/medtracker-migrate-jbfxr	0/1	Completed	0	2m42s
pod/medtracker-worker-54b6869fb9-hlzp	1/1	Running	0	2m42s

Рис 3.9.2. Состояние системы во время нагрузочного теста

Every 2.0s: kubectl get hpa,pods -n medtracker

MacBook-Pro-Vlad.local: Wed Jun 10 16:19:13 2026
in 0.082s (0)

NAME	REFERENCE	TARGETS	MINPODS	MAXPODS	REPLICAS	AGE
horizontalpodautoscaler.autoscaling/medtracker-api	Deployment/medtracker-api	cpu: 13%/70%, memory: 69%/90%	2	10	8	10m

NAME	READY	STATUS	RESTARTS	AGE
pod/medtracker-api-54dbcb95-2mccq	1/1	Running	0	10m
pod/medtracker-api-54dbcb95-9dpjc	1/1	Running	0	8m36s
pod/medtracker-api-54dbcb95-d7khl	1/1	Running	0	8m21s
pod/medtracker-api-54dbcb95-fk2m4	1/1	Running	0	10m
pod/medtracker-api-54dbcb95-r2zp2	1/1	Running	0	8m21s
pod/medtracker-api-54dbcb95-rp475	1/1	Running	0	10m
pod/medtracker-api-54dbcb95-tdj4l	1/1	Running	0	8m36s
pod/medtracker-api-54dbcb95-xxf62	1/1	Running	0	8m21s
pod/medtracker-worker-54b6869fb9-hlzp	1/1	Running	0	10m

Рис 3.9.3. Состояние системы после нагрузочного теста

ЗАКЛЮЧЕНИЕ

В рамках работы разработана сервисная платформа хранения данных о медицинских препаратах – серверная часть, обеспечивающая структурированное хранение справочных и персональных медицинских данных, их безопасную выдачу через программный интерфейс, а также надежность и масштабируемость при росте нагрузки.

В ходе решения первой задачи выполнен анализ предметной области и существующих решений. Определен состав хранимых данных – связный справочный каталог (диагнозы, препараты, добавки, побочные эффекты), персональные связи пользователя с каталогом и журналы самонаблюдения, выделены роли пациента и администратора и сформулированы функциональные и нефункциональные требования. Сравнительный анализ аналогов показал, что ни одна из этих категорий не закрывает совокупность поставленных задач и определил нишу разрабатываемой платформы как многопользовательской серверной системы.

В ходе решения второй задачи обоснован выбор архитектуры и технологического стека. Обоснован модульный монолит, организованный по принципам чистой архитектуры, как рациональное сочетание простоты развертывания и целостности данных с высокой сопровождаемостью и тестируемостью. Обоснованы программный интерфейс на основе gRPC поверх HTTP/2 с контрактами Protocol Buffers, реляционная база данных PostgreSQL с доступом через Entity Framework Core, а также распределенный кэш Redis как несущий элемент масштабируемости при нескольких репликах.

В ходе решения третьей задачи спроектированы модель предметной области и программный интерфейс. Разработана модель с древовидным справочным каталогом, связующими сущностями назначений пользователя, журналами самонаблюдения и иерархией базовых типов с аудитом и «мягким» удалением. Спроектирован программный интерфейс из восьми gRPC-сервисов с единой моделью ошибок на основе кодов состояния gRPC и сквозной трассировкой по идентификатору корреляции.

В ходе решения четвертой задачи реализованы прикладная логика, безопасность, доставка и кэширование. Реализованы сервисы аутентификации, каталога, персональных назначений и журналов, подсистема безопасности с токеной аутентификацией, хешированием паролей, защитой от перебора и атак по времени и безопасной обработкой одноразовых токенов с хранением только их хешей, доставка служебных писем на основе транзакционного Outbox с защитой от одновременного захвата сообщений несколькими репликами и выдержкой повторов, двухуровневое кэширование с версионированием каталога и распределенное ограничение частоты.

В ходе решения пятой задачи обеспечены контейнеризация и автомасштабирование. Приложение контейнеризировано, развертывание описано декларативными манифестами Kubernetes, управляемыми средствами kustomize. Реализовано горизонтальное автомасштабирование прикладного интерфейса от двух до десяти реплик по целевым показателям загрузки процессора и памяти, разделение ролей между репликами интерфейса и выделенной рабочей репликой, а также обновление без простоя по стратегии поэтапного обновления.

В ходе решения шестой задачи выполнено тестирование. Реализованы модульные и интеграционные тесты ключевых компонентов: сервиса аутентификации, обработчика Outbox, кэша каталога, ограничителя частоты, генератора токенов и сервиса очистки, а также разработана методика нагрузочного тестирования, подтверждающая работоспособность горизонтального автомасштабирования под нагрузкой.

Таким образом, совокупность решенных задач подтверждает достижение цели работы. Практическая значимость работы состоит в том, что предложенные архитектурные и инженерные решения образуют целостный образец построения платформ и могут быть использованы при проектировании аналогичных сервисов, а также в учебном процессе при изучении архитектуры серверных приложений.

Дальнейшее развитие платформы планируется в части практического применения. Поскольку сервис представляет собой серверную часть, его естественное дальнейшее использование – служить основой для клиентских приложений, через которые с платформой работают люди. На базе ее программного интерфейса может быть построено пользовательское приложение (мобильное или веб), в котором пациент ведет собственную медицинскую картину: просматривает справочный каталог, отмечает принимаемые препараты и добавки, фиксирует в журналах побочные эффекты и записи цикла и со временем накапливает историю самонаблюдения. Администратор при этом наполняет и сопровождает каталог через предусмотренные административные операции. Накопленные пользователем данные приобретают ценность за пределами самого приложения: с согласия пациента журналы могут выгружаться и передаваться лечащему врачу, давая ему более полную картину динамики состояния и приема препаратов для принятия решений, которые остаются за человеком, а не за системой. В таком виде платформа может использоваться как основа сервиса личного самоконтроля для пациентов.

СПИСОК ИСТОЧНИКОВ

1. Мартин, Р. Чистая архитектура. Искусство разработки программного обеспечения / Р. Мартин. – Санкт-Петербург : Питер, 2022. – 352 с.
2. Эванс, Э. Предметно-ориентированное проектирование (DDD). Структуризация сложных программных систем / Э. Эванс. – Москва : Вильямс, 2020. – 448 с.
3. Ньюмен, С. Создание микросервисов / С. Ньюмен. – Санкт-Петербург : Питер, 2023. – 624 с.
4. Клеппман, М. Высоконагруженные приложения. Программирование, масштабирование, поддержка / М. Клеппман. – Санкт-Петербург : Питер, 2026. – 640 с.
5. Ричардсон, К. Микросервисы. Паттерны разработки и рефакторинга / К. Ричардсон. – Санкт-Петербург : Питер, 2022. – 544 с.
6. Фаулер, М. Шаблоны корпоративных приложений / М. Фаулер. – Москва : Диалектика, 2020. – 544 с.
7. Хоп, Г. Шаблоны интеграции корпоративных приложений / Г. Хоп, Б. Вульф. – Москва : Диалектика, 2022. – 672 с.
8. Jones, M. RFC 7519: JSON Web Token (JWT) / M. Jones, J. Bradley, N. Sakimura ; Internet Engineering Task Force. – 2015. – URL: <https://www.rfc-editor.org/rfc/rfc7519> (дата обращения: 25.03.2026). – Текст : электронный.
9. Belshe, M. RFC 7540: Hypertext Transfer Protocol Version 2 (HTTP/2) / M. Belshe, R. Peon, M. Thomson ; Internet Engineering Task Force. – 2015. – URL: <https://www.rfc-editor.org/rfc/rfc7540> (дата обращения: 30.03.2026). – Текст : электронный.
10. gRPC Documentation : [сайт] / The gRPC Authors. – URL: <https://grpc.io/docs/> (дата обращения: 01.03.2026). – Текст : электронный.
11. Protocol Buffers Documentation : [сайт] / Google. – URL: <https://protobuf.dev/> (дата обращения: 01.03.2026). – Текст : электронный.

12. Kubernetes Documentation : [сайт] / The Kubernetes Authors. – URL: <https://kubernetes.io/docs/home/> (дата обращения: 22.04.2026). – Текст : электронный.

13. PostgreSQL 16 Documentation : [сайт] / The PostgreSQL Global Development Group. – URL: <https://www.postgresql.org/docs/> (дата обращения: 24.02.2026). – Текст : электронный.

14. Redis Documentation : [сайт] / Redis Ltd. – URL: <https://redis.io/docs/> (дата обращения: 05.04.2026). – Текст : электронный.

15. .NET documentation : [сайт] / Microsoft. – URL: <https://learn.microsoft.com/dotnet/> (дата обращения: 13.02.2026). – Текст : электронный.

16. Entity Framework Core documentation : [сайт] / Microsoft. – URL: <https://learn.microsoft.com/ef/core/> (дата обращения: 14.03.2026). – Текст : электронный.

17. gRPC services with ASP.NET Core : [сайт] / Microsoft. – URL: <https://learn.microsoft.com/aspnet/core/grpc/> (дата обращения: 01.03.2026). – Текст : электронный.

18. OWASP Top 10:2021 : [сайт] / OWASP Foundation. – URL: <https://owasp.org/Top10/> (дата обращения: 02.05.2026). – Текст : электронный.

19. Hangfire Documentation : [сайт]. – URL: <https://docs.hangfire.io/> (дата обращения: 08.05.2026). – Текст : электронный.

20. Российская Федерация. Законы. О персональных данных : Федеральный закон № 152-ФЗ : [принят Государственной Думой 8 июля 2006 года : одобрен Советом Федерации 14 июля 2006 года]. – Текст : электронный.

ПРИЛОЖЕНИЕ А

Исходный код доступен в репозитории на сервисе GitHub по ссылке:

<https://github.com/Olmec-a/MedTracker>

Или по QR-коду:

